NetPay Manager: manual técnico integral

Fase 3: Despliegue, seguridad, automatización y cierre
Empresa destinataria: Gravity
Sistema: administración interna de clientes, servicios de internet y cobros.

1. Propósito y alcance operativo

NetPay Manager centraliza el registro de clientes, planes de internet, empleados, pagos, obligaciones mensuales y recibos. Su objetivo es reemplazar el control disperso en hojas de cálculo por una operación transaccional y trazable. Los clientes son registros comerciales; las cuentas de acceso pertenecen a empleados.

La solución está formada por una aplicación Angular, una API REST y PostgreSQL. La API concentra la autorización y las reglas financieras. El navegador no accede directamente a la base de datos ni a credenciales de infraestructura.

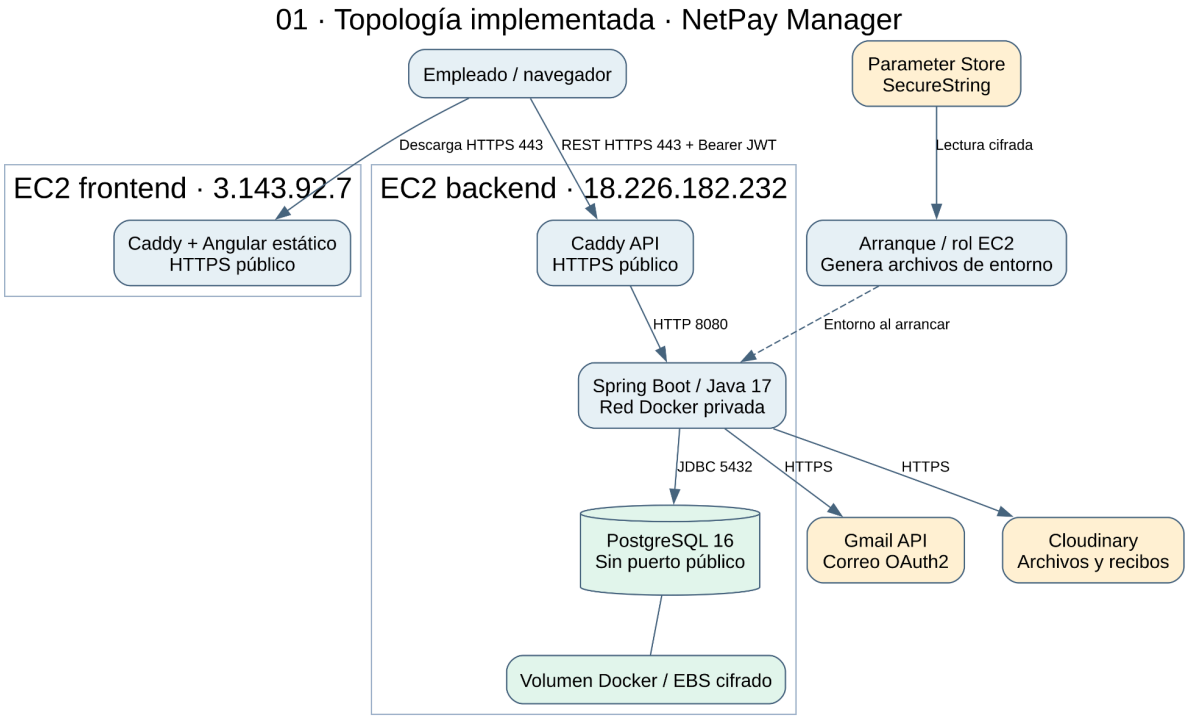
Este manual está dirigido a quien deba mantener, diagnosticar o evolucionar la solución. Explica responsabilidades, contratos, consistencia y dependencias; el diccionario físico y la matriz RF/HU se desarrollan en el documento de modelo de datos. Las fases 1 y 2 conservan el diseño y los criterios originales; esta revisión describe su materialización actual. Por ejemplo, el hosting final usa dos EC2 y PostgreSQL en Docker, en lugar de S3/CloudFront y RDS propuestos inicialmente.

Componente	Dirección pública	Responsabilidad
Aplicación web	https://3.143.92.7/#/auth/login	Acceso de administradores y cobradores
API REST	https://18.226.182.232/api	Operaciones y consultas del negocio
Verificación de disponibilidad	https://18.226.182.232/health	Disponibilidad de API y conexión SQL

Las direcciones corresponden a Elastic IP en `us-east-2`, región Ohio de AWS. Las rutas Angular utilizan fragmentos `/#/`; estos fragmentos no se transmiten al servidor HTTP.

2. Arquitectura implementada

La aplicación se implementa como un **monolito modular cliente-servidor**, con frontend y backend desplegados de manera independiente. La base de datos comparte instancia con la API, pero se ejecuta en un contenedor independiente dentro de una red privada de Docker.



La descarga de Angular y las llamadas REST son conexiones separadas del navegador. Caddy del frontend sirve archivos; no actúa como proxy de `/api`. Por ello, la API permite explícitamente el origen `https://3.143.92.7` mediante CORS.

2.1 Infraestructura y límites de disponibilidad

Recurso	**Backend**	**Frontend**
Instancia	`i-0e2bbaeb4cfd41f5d`	`i-0172012226e3d165f`
Tipo	`t3.small`	`t3.micro`
Sistema operativo	Amazon Linux 2023, x86_64	Amazon Linux 2023, x86_64

Recurso	Backend	Frontend
Disco raíz	gp3 cifrado, 24 GiB; retenido al terminar la instancia	gp3 cifrado, 12 GiB; eliminado al terminar la instancia
Contenedores	API, PostgreSQL, Caddy	Angular estático servido por Caddy
Red	VPC y subred pública propias	VPC y subred pública propias
Entrada autorizada	TCP 80 y 443	TCP 80 y 443
Administración	Systems Manager	Systems Manager

No hay RDS, balanceador, Auto Scaling ni distribución multi-AZ. La caída de la instancia backend afecta simultáneamente API y base de datos. La persistencia del disco ante reinicios no equivale a alta disponibilidad ni a respaldo. La arquitectura constituye una base modular para crecimiento, no una implementación de escalabilidad horizontal automática.

2.2 Tecnologías

Tecnología	Aplicación concreta
Angular 19 y TypeScript estricto	Componentes standalone, servicios HTTP, guards y rutas con carga diferida
Java 17 y Spring Boot 4.1.1	API y orquestación de casos de uso
Spring Security y JJWT 0.12.6	Autenticación y autorización
Spring Data JPA / Hibernate	Adaptación de persistencia y validación del esquema
PostgreSQL 16	Integridad referencial, transacciones y agregados de reportes
Flyway	Migraciones versionadas V1–V21
Apache POI 5.4.1	Lectura de hojas de cálculo y exportación binaria XLS
Flying Saucer / OpenPDF	Conversión de plantillas HTML a PDF

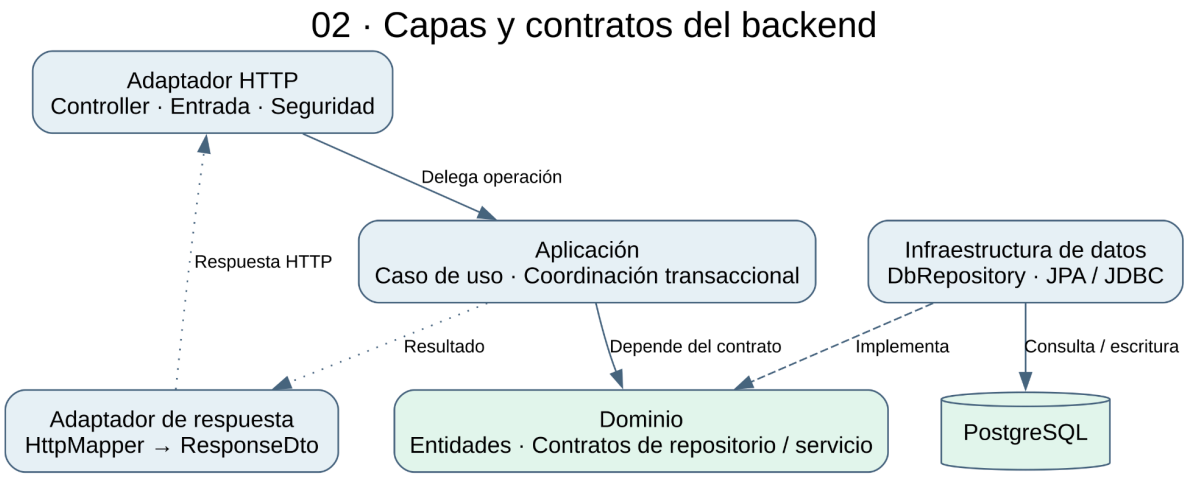
Tecnología	Aplicación concreta
Caddy 2.11.7	Terminación TLS, certificados ACME y servicio de archivos estáticos
Docker Compose	Topología de contenedores y comprobaciones de disponibilidad
Terraform	Aprovisionamiento declarativo de infraestructura
GitHub Actions	Verificación, construcción, publicación y despliegue
AWS ECR / SSM / IAM	Registro privado de imágenes, ejecución remota y permisos

3. Organización del backend

Raíz de código: `src/main/java/com/navi/net_pay_backend/`.

Capa	Estructura	Responsabilidad
Dominio	`domain/entity`, `repository`, `service`, `dto`, `enums`, `exception`	Entidades, contratos, resultados y reglas independientes de persistencia
Aplicación	`application/use_case`, `application/model`	Coordinación de operaciones, validación de negocio y límites transaccionales
Infraestructura	`infrastructure/db`	Entidades JPA, consultas e implementaciones `Db*Repository`
Infraestructura HTTP	`infrastructure/http/controller`, `dto`, `mapper`, `middleware`	Contratos REST, mapeo y manejo de errores
Seguridad y servicios	`infrastructure/security`, `service`, `config`	JWT, contexto de seguridad, correo, documentos y configuración

Las entidades de dominio no tienen anotaciones JPA. Las entidades de persistencia utilizan el sufijo `Entity`; los adaptadores de repositorio traducen entre ambas representaciones. La aplicación utiliza servicios Spring para coordinar los casos de uso. Esta separación corresponde a una arquitectura por capas inspirada en Clean Architecture, con infraestructura dependiente de los contratos internos.



El endpoint `/health` realiza una consulta mínima `SELECT 1` mediante JDBC y no expone estructura, versión ni credenciales de la base.

3.1 Límites de responsabilidad y mantenimiento

Una operación HTTP atraviesa autenticación, autorización, adaptación de entrada, caso de uso y persistencia. El contrato del repositorio permite que la aplicación solicite una operación sin conocer una tabla o un proveedor. JPA resuelve mantenimiento de entidades; JDBC y SQL agregados se utilizan en reportes y sincronización masiva de obligaciones. Cambiar un agregado financiero exige revisar ambas rutas de acceso, no únicamente el formulario que lo muestra.

El diseño objetivo emplea DTO de escritura separados del dominio. La implementación todavía conserva controladores de clientes/pagos que reciben entidades de dominio; por ello, la separación de entrada no es uniforme. Al mantener estos recursos se debe inventariar el JSON realmente consumido por Angular y migrarlo de forma compatible. La estructura de capas es una guía de responsabilidad, no una garantía de que todos los límites hayan sido aislados por completo.

Tipo de modificación	Punto principal de análisis	Dependencias que deben revisarse
Regla de cartera o pago	Caso de uso y contratos del dominio	Aplicaciones, recibo, auditoría, importación y agregados SQL
Campo persistente	Modelo de datos y adaptadores	Nueva migración, validación Hibernate, DTO y formulario
Consulta lenta	Repositorio y plan de ejecución SQL	Índices, filtros, volumen real y número de consultas por resultado
Servicio externo	Contrato del dominio y adaptador	Configuración, fallos HTTP y efectos que no revierte PostgreSQL
Nuevo permiso o recurso	Catálogo y autorización de API	Rutas Angular, menú, controles y pruebas de alcance

3.2 Transacciones y fallos

El límite transaccional se establece en las operaciones que coordinan escrituras relacionadas, especialmente pagos, recibos y desafíos de acceso. Si falla una escritura participante, se revierte la transacción SQL. Una llamada Gmail o una carga Cloudinary ocurre fuera del control transaccional del motor: su efecto remoto puede permanecer aunque la transacción local falle. El sistema actual no dispone de outbox ni de compensación distribuida.

Los errores de negocio se traducen mediante el middleware global; la cadena de seguridad también genera sus propias respuestas 401/403. Durante soporte debe conservarse la clave de error y el estado HTTP: no existe un único envoltorio JSON uniforme para todos los fallos.

4. Organización del frontend

El proyecto `net-pay-frontend` contiene:

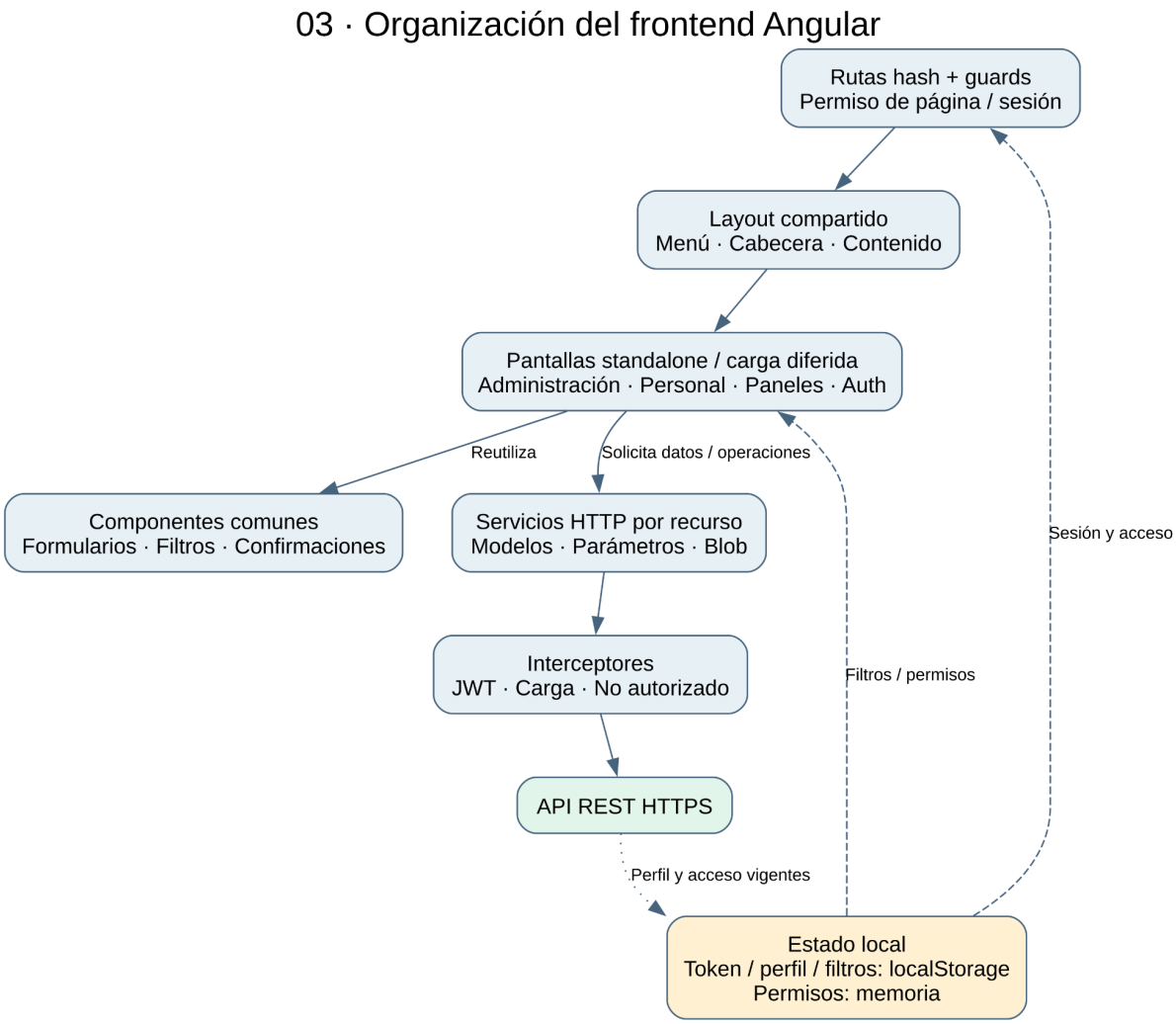
- `src/app/features`: pantallas agrupadas en autenticación, administración, dashboard y operaciones personales.
- `src/app/services/core`: acceso a la API por recurso.
- `src/app/security`: guards de navegación y autorización de rutas.
- `src/app/interceptors`: manejo transversal de carga y respuestas no autorizadas.
- `src/app/commons`: componentes y utilidades reutilizables.

- `src/app/model`: modelos y DTO utilizados en la interfaz.
- `src/environments`: configuración de desarrollo y producción.

`environment.prod.ts` configura `baseUrl=https://18.226.182.232/api` y `backendDomain=18.226.182.232`. El dominio permitido por el interceptor JWT debe coincidir con el destino de la API para adjuntar `Authorization`.

El token y el perfil se conservan en `localStorage` con las claves del enum `AppStorageKeys`. Los permisos se consultan mediante `/api/auth/access`; la interfaz no deduce autorizaciones a partir del nombre del rol. Los guards mejoran la experiencia de navegación, pero la autorización efectiva se realiza en el backend.

4.1 Navegación, estado y comunicación



Las rutas agrupan autenticación, administración, paneles y páginas personales. El layout común aloja menú, cabecera y contenido; las pantallas se cargan de forma diferida. El guard consulta la sesión y los permisos necesarios para la ruta, mientras que la directiva de permiso gobierna acciones concretas, como editar o exportar. El inicio después del login se elige entre las páginas accesibles; no presupone que todo empleado tenga acceso al panel administrativo.

Los servicios por recurso encapsulan URL, parámetros y tipo de respuesta. Las pantallas coordinan la interacción, los formularios y el resultado. El estado de listados conserva filtros, orden y paginación en almacenamiento local y los transforma a parámetros HTTP. Un filtro recordado puede explicar una lista aparentemente vacía después de volver a una pantalla.

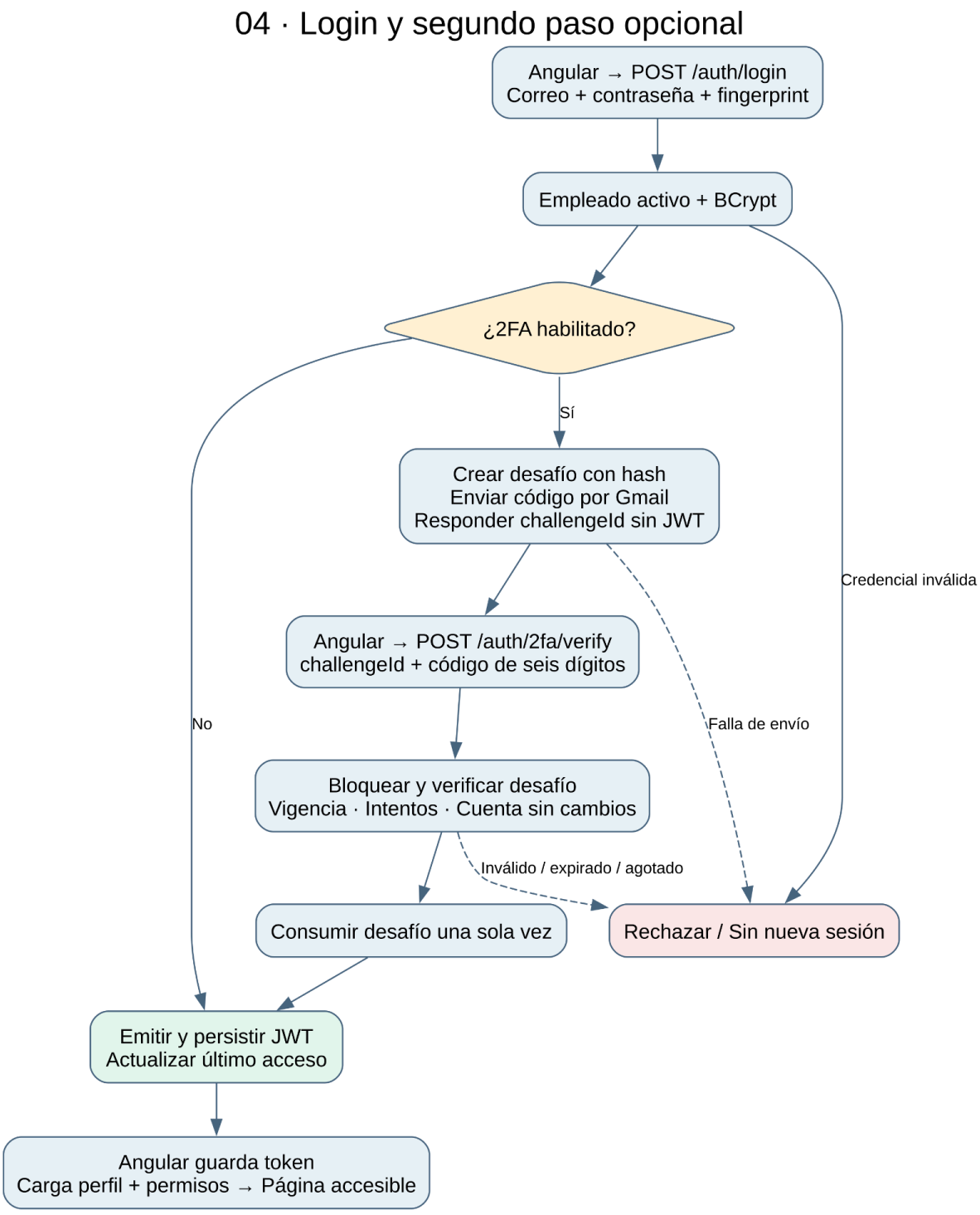
Los interceptores gestionan carga global y respuestas no autorizadas. Las exportaciones omiten la capa global de carga y manejan su estado en el botón; reciben un `Blob`, no JSON. La actualización de perfil/permisos usa caché en memoria y comparte solicitudes en curso. El layout refresca permisos periódicamente y al recuperar el foco; el servidor los recalcula al recibir cada solicitud protegida.

4.2 Criterios de mantenimiento del cliente

- Conservar el patrón de filtros y servicios compartidos al incorporar una lista; esto evita que una pantalla interprete `offset` de forma diferente de otra.
- Mantener coherentes formulario, modelo de entrada y contrato HTTP. Validar en navegador ayuda a la interacción, pero no sustituye las validaciones del servidor.
- Distinguir datos de negocio de estado visual. El perfil local puede estar desactualizado; no es la fuente de autorización ni de saldos.
- Revisar ruta, permiso de página, permiso de acción y menú al incorporar una función. Ocultar un botón no restringe la API.
- Al cambiar la dirección de la API, reconstruir Angular y revisar `baseUrl`, `backendDomain`, CORS y enlaces de recuperación. La configuración del bundle es de compilación; reiniciar su contenedor no la reemplaza.
- Verificar tipos y plantillas con `npm run build`; el repositorio no define un comando de lint. Las dependencias instaladas no equivalen a módulos funcionales incorporados.

5. Identidad y seguridad

5.1 Flujo de autenticación



El login recibe correo, contraseña y fingerprint. Primero busca un empleado activo y compara la contraseña con BCrypt. Si `twoFactorEnabled` es falso, emite el token; si es verdadero, responde con un desafío público y vencimiento, sin JWT ni actualización de último acceso. Angular presenta entonces el formulario de seis dígitos y solo carga perfil/permisos después de verificar el código.

La firma y expiración del JWT se verifican junto con su existencia vigente en `token_credential`. El filtro comprueba además el estado del usuario y calcula permisos actuales. Por tanto, la API es sin sesión HTTP de servidor, pero la autenticación sí depende de consultas a PostgreSQL; no es una validación exclusivamente criptográfica y sin estado persistente.

Las contraseñas se almacenan con BCrypt. El fingerprint identifica la credencial emitida para un dispositivo y permite sustituir la sesión previa de ese usuario/fingerprint; no representa un segundo factor de autenticación.

La duración predeterminada del JWT es 3 600 segundos, configurable. No se implementa un flujo de refresh token de sesión. El cierre de sesión de Angular limpia almacenamiento y caché local; no llama a un endpoint de revocación. En consecuencia, una copia del token emitido puede seguir siendo válida hasta su vencimiento o invalidación persistente. Desactivar al empleado sí impide el uso posterior porque su estado se comprueba en cada solicitud. Tampoco debe asumirse que cambiar la contraseña elimina automáticamente todas sus sesiones existentes.

5.2 Verificación en dos pasos por correo

La preferencia puede gestionarse en mantenimiento de empleados y en el perfil propio. V21 añade la bandera y `login_challenge`; los empleados existentes parten con la bandera desactivada. El segundo paso implementado es un código por correo, no TOTP ni una aplicación autenticadora.

Control	**Comportamiento implementado**	**Consecuencia operativa**
Código	Seis dígitos aleatorios, almacenados como hash	Tratarlo como texto para conservar ceros iniciales
Vigencia	Cinco minutos por defecto	El servidor decide el vencimiento; `expiresAt` no incluye zona
Intentos	Hasta cinco; los fallidos persisten	Volver a iniciar el acceso al agotar el desafío

Control	Comportamiento implementado	Consecuencia operativa
Reenvío	Nueva solicitud de login; espera mínima de 60 segundos por empleado	El límite es de emisión de código, no un rate limit global de la API
Reutilización	Consumo único y bloqueo transaccional del desafío	Dos verificaciones concurrentes no deben emitir dos sesiones por el mismo desafío
Cambios de cuenta	Rechazo si cambió correo/contraseña, se desactivó 2FA o el empleado ya no está activo	Solicitar un desafío nuevo después de modificar la cuenta
Falla de Gmail	Revierte el desafío y no emite sesión	El correo pasa a ser dependencia del login de las cuentas con 2FA

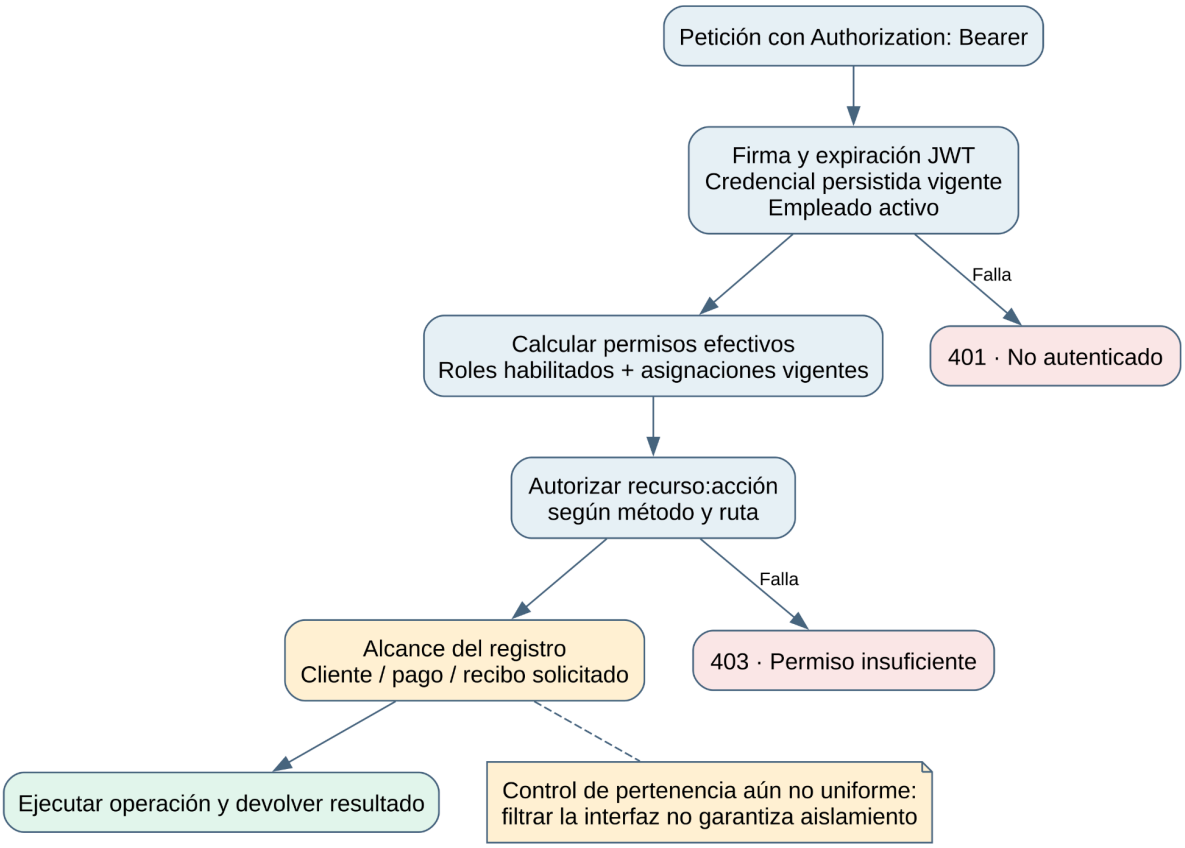
Activar o desactivar la preferencia afecta los próximos inicios; no fuerza el cierre de sesiones abiertas. La disponibilidad del envío debe verificarse antes de habilitarla a los empleados que operan el sistema.

5.3 Autorización y alcance de datos

Los permisos tienen la forma `recurso:acción`. Las asignaciones de rol incluyen inicio, vencimiento opcional y cancelación. La autorización toma la unión de permisos de asignaciones efectivas. Los roles base Administrador y Cobrador están `LOCKED`: son asignables, pero no editables ni eliminables por las operaciones normales.

La lectura, creación, modificación, eliminación, importación y exportación son acciones independientes. Por ejemplo, `reports:read` permite consultar y `reports:export` permite descargar; tener uno no implica tener el otro. Los permisos personales habilitan rutas compartidas y vistas filtradas por cobrador. Esto exige comprobar también la pertenencia del recurso: la autorización de una ruta no demuestra por sí sola que el cliente o pago solicitado corresponda al actor.

05 · Controles de una solicitud protegida



Los permisos efectivos provienen de asignaciones cuyo inicio ya ocurrió, cuyo fin no venció y que no fueron canceladas, sobre roles habilitados. Cancelar una asignación conserva su historial. El JWT identifica al empleado; no se utiliza una lista fija de roles del token como autoridad duradera.

Existe deuda técnica en el aislamiento por registro: el alta de pago como cobrador fija el actor autenticado, pero no verifica allí la asignación del cliente; las consultas directas por identificador de pagos tampoco incorporan uniformemente una comprobación de pertenencia. La aceptación de cambios de acceso debe incluir solicitudes directas de un cobrador sobre recursos de otro, además de la navegación normal. Esta revisión documental no considera el filtrado visual una garantía completa de aislamiento.

5.4 Recuperación y servicios externos

La recuperación utiliza un token aleatorio, almacena su hash y su vencimiento y elimina ambos valores al completar el cambio. La duración predeterminada es 30 minutos. Los enlaces se construyen a partir de `FRONTEND_BASE_URL` y la ruta `/reset-password`.

El flujo público de recuperación por correo convive con el cambio de contraseña del perfil. Es una evolución frente al control exclusivamente administrativo planteado en las primeras fases. La nueva contraseña de recuperación exige al menos ocho caracteres y confirmación. Para hash routing, `FRONTEND_BASE_URL` debe incluir el fragmento adecuado (por ejemplo, `https://3.143.92.7/#`); se debe abrir el enlace real recibido para verificar el resultado completo.

Gmail API utiliza OAuth2 con refresh token; no se usa SMTP. Cloudinary proporciona almacenamiento externo de archivos y comprobantes. La presencia del código de integración no acredita por sí sola entrega de correo o carga de archivos en producción: ambas requieren credenciales operativas y pruebas de extremo a extremo.

5.5 Seguridad de infraestructura y navegador

- HTTPS público gestionado por Caddy con certificados Let's Encrypt para IP y perfil `shortlived`; vigencia aproximada de 160 horas y renovación automática.
- `default_sni` selecciona el certificado de la IP para clientes sin SNI.
- Discos EBS cifrados; acceso SQL limitado a la red Docker, sin publicación de 5432.
- Security Groups sin entrada SSH ni acceso directo a Spring Boot.
- Rol de instancia backend para leer la imagen ECR y el SecureString concreto.
- Usuario de automatización `netpay-github-actions`, compartido por ambos repositorios, con políticas separadas de ECR/SSM para cada componente.
- Claves persistentes de automatización en GitHub Secrets; configuración de aplicación en Parameter Store. Terraform no administra el valor del SecureString ni la clave secreta del usuario de despliegue.

CORS autoriza orígenes del navegador; no autentica empleados ni limita clientes HTTP externos. Se exponen `Authorization`, `X-Total-Count` y `Content-Disposition` para sesión, paginación y descargas. La API utiliza Bearer explícito y sesiones HTTP deshabilitadas; la configuración actual deshabilita CSRF. El token en `localStorage` es accesible a JavaScript del origen: mantener dependencias y renderizado de contenido es parte del mantenimiento de la sesión. HTTPS protege el tránsito público, pero no convierte el almacenamiento del navegador en un almacén de secretos de servidor.

6. Procesos de negocio y contratos HTTP

Proceso	**Entrada principal**	**Implementación**
Identificación del empleado	`POST /api/auth/login`	`LoginUseCase`, `TokenCredentialUseCase`
Perfil y permisos efectivos	`GET /api/users/me`, `GET /api/auth/access`	`AppUserUseCase`, `AccessControlUseCase`

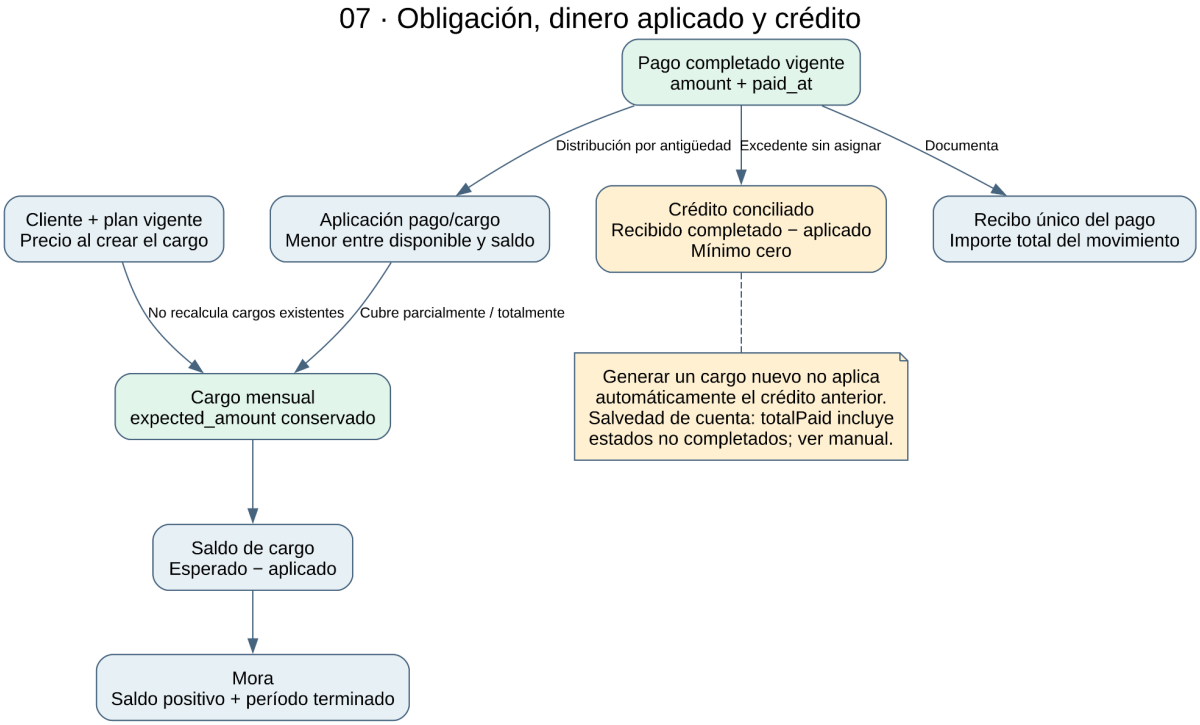
Proceso	Entrada principal	Implementación
Clientes y planes	`/api/clients`, `/api/services`	`AppClientUseCase`, `AppServiceUseCase`
Cobro administrativo o personal	`POST /api/payments`, `POST /api/payments/collector`	`AppPaymentUseCase`
Cuenta y obligaciones	`/api/clients` y recursos de cuenta del cliente	`ClientAccountUseCase`, `ClientChargeGenerator`
Comprobantes	`/api/receipts`, `/api/payments/{hashId}/receipt/pdf`	`ReceiptUseCase`
Indicadores	`/api/dashboard`, `/api/dashboard/collector`	`DashboardUseCase`
Reportes	`/api/reports/collections`, `/api/reports/overdue-clients`	`ReportUseCase`
Incorporación histórica	`POST /api/imports/{resource}`	`DataImportUseCase`, `UserPaymentImportHandler`
Avisos de mora	`GET /api/notifications`	`OverdueNotificationUseCase`
Bitácora	`GET /api/audit-logs`	`AuditLogUseCase`

El identificador público de recursos es `hashId` y se usa en las rutas CRUD. Hay contratos heredados que todavía transportan IDs numéricos en relaciones o filtros, y el JWT incluye `userId`; al mantener una integración se debe consultar el contrato concreto, sin confundirlo con el identificador público de la URL. La evolución debe unificar esas relaciones hacia identificadores públicos. Los parámetros de tipología utilizan `internal_id`, no `typology_id`. Las listas paginadas emplean `offset` como desplazamiento de registros, `size` y, cuando corresponde, la cabecera `X-Total-Count`.

Los errores controlados se expresan como claves breves. Las respuestas distinguen 400 para entrada o regla inválida, 401 para falta de autenticación, 403 para falta de permiso, 404 para entidad inexistente, 409 para conflicto y 500 para errores inesperados.

7. Modelo financiero, consistencia y conciliación

7.1 Conceptos y responsabilidades



Concepto	**Qué representa**	**Qué no debe inferirse**
Plan	Condiciones y precio vigente del servicio	Un historial completo de tarifas contratadas
Período	Mes calendario con fecha inicial y final	La fecha de recepción de dinero
Cargo	Obligación de un cliente en un período, con importe conservado	Un pago ni una factura fiscal

Concepto	Qué representa	Qué no debe inferirse
Pago	Movimiento recibido con fecha efectiva, importe, método y estado	Que el banco haya procesado una transacción
Aplicación	Parte de un pago que cubre un cargo	Que todo el pago pertenezca a un único mes
Recibo	Comprobante único del pago completado	Detalle exhaustivo de todos los cargos cubiertos

El cargo mensual representa la obligación y `payment_allocation` representa cuánto de un pago cubre esa obligación. Solo un pago completado participa en la aplicación de abonos. Los pagos pendientes o fallidos no cancelan deuda.

Los cargos existentes conservan `expected_amount`. Si se modifica el precio actual del plan, los cargos previamente creados no se recalculan. La creación retrospectiva de cargos utiliza el precio disponible al generarlos; no existe una tabla histórica de tarifas que reconstruya automáticamente precios antiguos.

Un pago se aplica a cargos con saldo pendiente, en orden de período. Un excedente puede quedar como saldo a favor; no implica por sí mismo crear todos los cargos de meses futuros. Los recibos se emiten para pagos completados y un pago con recibo puede quedar bloqueado para edición. La anulación elimina asignaciones y marca el pago como eliminado lógicamente, conservando auditoría.

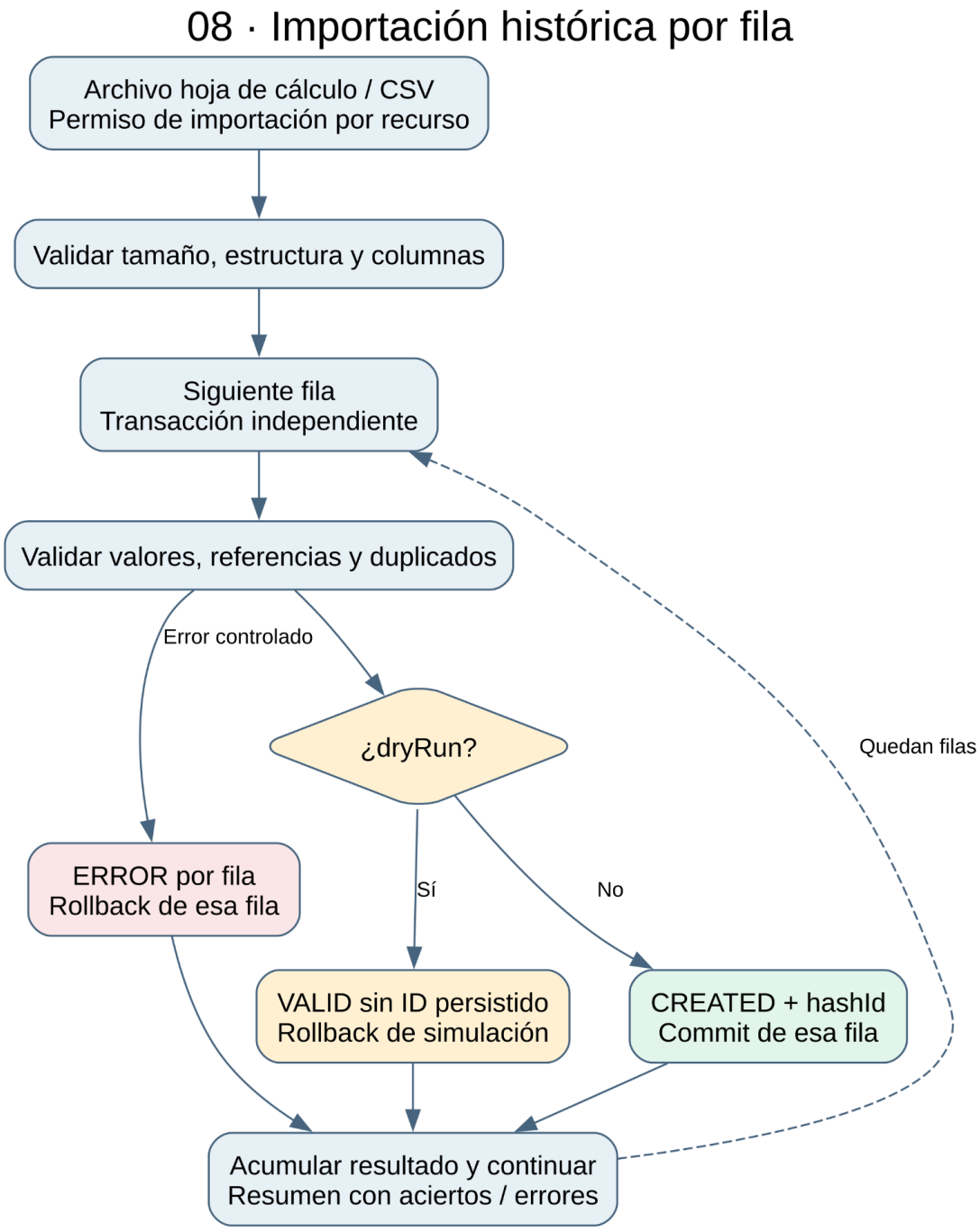
7.2 Generación de obligaciones y aplicación del dinero

La generación recorre desde el mes de alta comercial del cliente hasta el mes actual o la fecha del pago, si esta es futura. Si falta la fecha de alta se toma la fecha actual. Los cargos ya existentes se reutilizan. Las consultas de cuenta y ciertas consultas agregadas también aseguran obligaciones faltantes; por tanto, algunas lecturas funcionales pueden ocasionar escrituras de sincronización. No hay un proceso programado independiente que garantice haber generado toda la cartera antes de la primera consulta.

El alta valida importe positivo, fecha efectiva y método; si no se proporciona estado, el pago queda pendiente. Al completarlo, se distribuye contra los cargos más antiguos con saldo. En cada cargo se aplica el menor valor entre el dinero disponible y la obligación aún descubierta. La fecha efectiva del pago y la fecha técnica de registro son conceptos distintos; los reportes de cobros usan la primera.

Ejemplo de conciliación: dos cargos de Q150 suman Q300. Un pago completado de Q220 aplica Q150 al primero y Q70 al segundo: quedan Q80 pendientes y cero crédito. Un segundo pago de Q100 aplica Q80 y deja Q20 sin asignar. El total recibido es Q320, el aplicado Q300 y el crédito Q20. Si el mismo Q220 se registra pendiente, no cubre los cargos ni genera recibo.

7.3 Saldos



Magnitud	Cálculo e interpretación
Saldo de cargo	Importe esperado menos aplicaciones del cargo
Saldo de cuenta	Suma de importes esperados menos suma aplicada
Crédito mostrado por la cuenta	Máximo entre cero y `totalPaid` menos importe aplicado; revisar la salvedad de estados descrita a continuación
Estado visible	Pendiente, parcial, pagado o vencido según aplicaciones y fecha de cierre calendario

Deuda y crédito se muestran por separado; el saldo no se obtiene simplemente restando todos los pagos a todos los cargos. La generación de un cargo posterior no redistribuye automáticamente el crédito previo. El modelo requiere una evolución explícita si se desea esa compensación automática. El mes corriente puede tener saldo pendiente sin estar vencido.

Salvedad del cálculo actual de cuenta: la suma que alimenta `totalPaid` excluye el estado eliminado, pero no restringe a completados. Puede incluir pendientes o fallidos y, por tanto, inflar el crédito mostrado aunque esos pagos no hayan generado aplicaciones. Para conciliación financiera se debe calcular por separado el recibido completado vigente. Alinear esa suma con el criterio financiero es un ajuste pendiente de implementación; no debe interpretarse ese crédito visual como dinero confirmado sin revisar los estados.

7.4 Edición, anulación y recibos

La edición elimina las aplicaciones anteriores y vuelve a ejecutar la distribución dentro de la transacción. Un pago completado que ya tiene recibo se rechaza para edición con `payment_locked_by_receipt`. La anulación retira aplicaciones, marca el movimiento eliminado y registra la acción; no representa una devolución bancaria ni un borrado del historial.

El recibo tiene número único y relación uno-a-uno con el pago. Si Cloudinary está configurado, puede almacenarse el PDF durante su emisión; en caso contrario se renderiza al solicitarlo. Si ya hay archivo almacenado y el proveedor está configurado, se descarga desde allí. Marcar impresión conserva una fecha y auditoría: no confirma que una impresora física haya entregado papel.

El período del recibo se toma como referencia de un cargo con aplicaciones del cliente; no constituye el detalle completo de la distribución de ese pago. La conciliación por período se realiza con las aplicaciones, no solo con el encabezado del recibo. Al anular se debe verificar también cómo se presentan los comprobantes históricos; no existe un ciclo fiscal independiente de cancelación del recibo.

7.5 Integridad bajo concurrencia

Las restricciones únicas evitan duplicar cliente/período, pago/cargo y recibo/pago. No garantizan que dos pagos distintos, procesados simultáneamente, no lean el mismo saldo y sobreasignen un cargo. La implementación de aplicaciones no incorpora un bloqueo o versionado que resuelva por sí solo ese escenario. Antes de introducir múltiples escritores o réplicas se requiere ese control y pruebas de concurrencia. Una transacción aislada correctamente no equivale a serialización de todos los cobros.

Para investigar una diferencia se sigue cliente → cargos → pagos completados vigentes → aplicaciones → recibos → auditoría. Deben compararse fecha efectiva, fecha de corte, filtros, anulaciones y plan conservado en el cargo. Cambiar el precio del plan o eliminar registros para hacer coincidir una pantalla puede destruir la explicación del saldo.

8. Datos, importaciones y salidas documentales

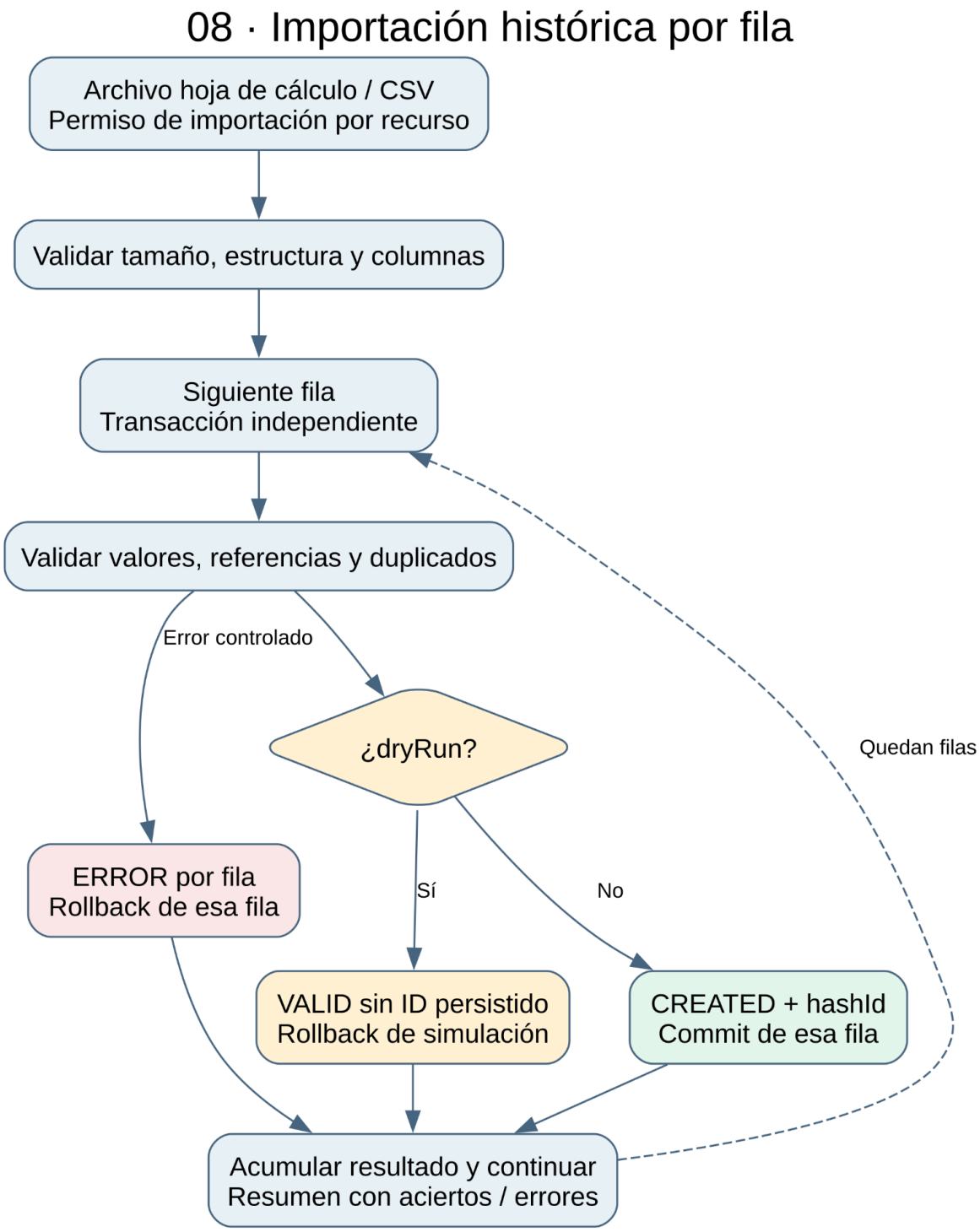
8.1 Organización y evolución del modelo

El modelo separa identidad/permisos, cartera y contabilidad. El modelo de datos y trazabilidad contiene la figura entidad-relación, diccionario, restricciones e historial de versiones. V21 incorpora desafíos de login y la preferencia 2FA. La aplicación verifica el esquema con Hibernate después de que Flyway aplique las migraciones pendientes al arrancar.

Los estados comerciales permiten bajas lógicas; las tablas de aplicaciones y relaciones pueden requerir eliminaciones físicas de mantenimiento dentro de la operación. La bitácora usa una referencia lógica al identificador de la entidad, no una FK universal. Mantener una relación no significa que su dato descriptivo sea inmutable: nombres y datos de contacto pueden cambiar, mientras que el importe esperado del cargo ya creado sí se conserva.

Un cambio de esquema debe definir compatibilidad con la imagen anterior, transformación de datos y validación posterior. Se añade una versión nueva; no se reescribe una aplicada. Las migraciones aditivas facilitan expansión/contracción; una eliminación de columna puede impedir rollback de binario. La evidencia aplicada es `flyway_schema_history`, no solo la presencia del archivo en Git.

8.2 Importación histórica



El lector admite hojas de cálculo y CSV con columnas esperadas por recurso. El límite multipart efectivo configurado es 5 MiB por archivo y 6 MiB por solicitud. Se validan estructura, valores y referencias; servicios y empleados deben estar disponibles antes de cargar clientes y pagos que dependan de ellos. La coincidencia por nombre/correo requiere resolver ambigüedades, no escoger arbitrariamente una persona.

Cada fila se procesa en una transacción independiente. El resultado identifica filas creadas, válidas en simulación o con error; un error de fila controlado no revierte las anteriores. Un fallo inesperado de la solicitud puede ocurrir después de confirmar filas, de modo que el reintento debe partir del resultado y la conciliación, no de suponer que todo el archivo fue rechazado.

`dryRun` ejecuta validación y marca rollback por fila; no devuelve un identificador persistido ni almacena el PDF externo del pago simulado. No debe interpretarse como ausencia absoluta de efectos técnicos: las secuencias SQL pueden consumir números aunque la fila se revierta. En pagos, `import_key` y su índice único permiten identificar duplicados persistentes. No existe una cabecera de lote persistida que permita reanudar cualquier importación de forma automática.

8.3 Reportes, paneles y notificaciones

Los paneles presentan agregados; los reportes de cobros y mora usan consultas específicas sobre pagos, cargos y aplicaciones. Los cobros por fecha efectiva no son equivalentes a la deuda por cierre de período. Un cambio de fórmula debe revisarse en consulta, exportación y panel relacionado para evitar definiciones contradictorias.

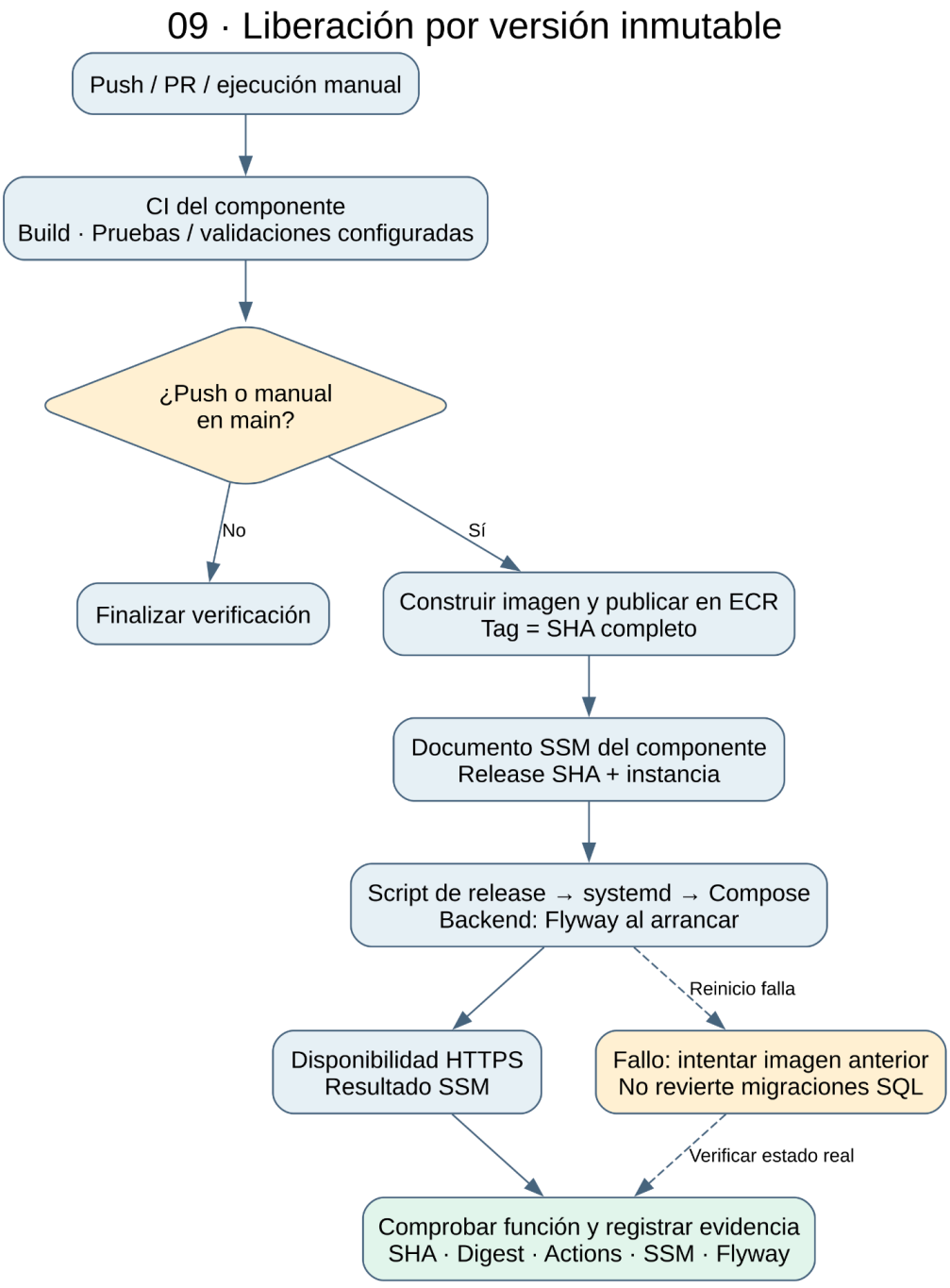
La exportación actual es síncrona y admite PDF y **XLS binario**, con celdas numéricas para importes/cantidades y separación en hojas cuando se alcanza el límite del formato. `xls` es el formato predeterminado de API; `csv` y `xlsx` se rechazan. Angular descarga un `Blob` y puede leer `Content-Disposition` gracias a CORS. Debe verificarse el valor enviado por la pantalla: aún existen referencias CSV en el frontend que requieren alineación con el contrato vigente.

La consulta paginada de mora y su exportación tienen propósitos distintos: el archivo contiene el conjunto filtrado, no necesariamente la página visible. Una exportación grande consume memoria y tiempo del proceso HTTP; no hay cola de trabajos ni notificación persistida de finalización. Las notificaciones de mora se calculan al consultar y respetan el alcance definido para la consulta; no constituyen un historial de mensajes enviados a clientes.

Las fórmulas y criterios de conciliación se desarrollan en reportes y validación; para el contrato actualizado de XLS/2FA se consulta también API de integración.

9. Configuración, automatización y trazabilidad técnica

Los repositorios son NaviM02/net-pay-backend y NaviM02/net-pay-frontend. Los workflows diferencian verificación de publicación; `main` es la rama de producción. Un despliegue requiere verificación exitosa y emplea el SHA completo del commit como tag inmutable de imagen.



Terraform se ejecuta con una identidad de aprovisionamiento separada de la identidad usada por Actions. El workflow despliega imágenes, no ejecuta `terraform apply`. Flyway evoluciona la base al arrancar la nueva versión de la API. El historial `flyway_schema_history`, los logs de Actions, el tag ECR, el comando SSM y la bitácora funcional cubren aspectos diferentes de trazabilidad y deben conservarse como evidencia independiente.

9.1 Configuración por ambiente

Configuración	**Dónde se resuelve**	**Cuándo requiere intervención**
URL API y dominio JWT de Angular	Bundle de producción	Reconstruir y desplegar frontend si cambian
Base SQL y firma/vigencia JWT	Variables de aplicación	Actualizar origen seguro y reiniciar API; rotar firma invalida tokens anteriores
Origen CORS y URL de recuperación	Variables de aplicación	Revisar navegador, preflight y enlace enviado
Gmail y Cloudinary	Secretos/configuración de proveedores	Probar la operación específica después del cambio
EIP, roles IAM, redes y discos	Terraform	Revisar plan, estado y efectos de reemplazo
SHA y repositorio de imagen	Configuración de release por host	Conservar versión anterior y digest para rollback

El arranque backend lee el SecureString mediante el rol EC2 y genera `app.env` y `db.env` con permisos restringidos en `/etc/netpay`, donde también permanece la configuración de despliegue. `/run/netpay` se utiliza para archivos temporales de autenticación Docker. Parameter Store no inyecta dinámicamente valores en un proceso ya iniciado: el arranque debe volver a obtenerlos. Cambiar la contraseña de conexión tampoco cambia automáticamente la contraseña de un PostgreSQL ya inicializado; esa rotación requiere una operación coordinada sobre la base y el consumidor.

El escritor de entorno admite una lista explícita de claves. Actualmente no incluye `JWT_EXPIRATION` ni `TWO_FACTOR_EXPIRATION_MINUTES`: esas opciones existen en Spring, pero no pueden añadirse sin más al JSON de producción, porque la validación lo rechazaría.

Se deben actualizar y desplegar de forma coordinada el escritor y su configuración, o conservar los valores predeterminados.

9.2 Diagnóstico y evidencia

La bitácora funcional identifica actor, acción, recurso, fecha y cambios. Los logs de aplicación explican ejecución y fallos; Flyway explica evolución del esquema; Actions/ECR/SSM explican qué se publicó y ejecutó. Ninguna de estas fuentes sustituye las demás. El sistema no incorpora un identificador de traza distribuida universal que una automáticamente todas las fuentes.

Para una discrepancia de usuario se registra el identificador público del recurso y el momento; para una liberación, ambos SHA (frontend/backend), digest, ejecución Actions, comando SSM y versión SQL. Se conservan evidencias sin tokens, contraseñas ni datos personales innecesarios. El endpoint de salud acredita proceso y conectividad SQL, no exactitud financiera o entrega de correos.

10. Mantenimiento y validación de cambios

10.1 Ruta de análisis según el síntoma

Pregunta de mantenimiento	**Dónde comenzar**	**Qué contrastar**
¿Por qué no aparece una página o acción?	Acceso efectivo y ruta Angular	Asignaciones vigentes, permiso de página/acción y respuesta 403
¿Por qué un saldo no coincide?	Cuenta y aplicaciones del cliente	Fecha de alta/corte, estado del pago, anulaciones y cargo conservado
¿Por qué cambió un cliente o pago?	Auditoría por recurso	Actor, instante, cambios y versión desplegada
¿Por qué no descarga un reporte?	Solicitud y respuesta HTTP	Formato XLS/PDF, permiso de exportación, Blob y CORS
¿Por qué falla el login con 2FA?	Desafío y proveedor Gmail	Intentos, vencimiento, cambio de cuenta y entrega del mensaje
¿Por qué falla tras desplegar?	SSM/systemd y logs de arranque	Imagen real, configuración, historial Flyway y estado SQL

10.2 Verificación por impacto

El backend se compila con el wrapper Maven (`./mvnw clean compile`) y sus pruebas se ejecutan con `./mvnw test`; las verificaciones de integración que usan PostgreSQL deben disponer de su entorno de base/Docker. El frontend se valida con `npm run build` y las pruebas configuradas para las funciones afectadas. Compilar no confirma migraciones aplicadas ni comportamiento del despliegue.

Cambio	**Validación funcional mínima**
Identidad/permisos	Login sin/con 2FA, código incorrecto/expirado/reutilizado, empleado inactivo, permiso revocado y acceso directo ajeno
Pagos/cargos	Parcial, completo, excedente, pendiente, anulación, edición bloqueada y concurrencia si se modifica distribución
Tarifas	Cargo existente conserva importe; cargo nuevo usa tarifa vigente; no reconstruir pasado sin fuente
Importación	Simulación, fila inválida, duplicado persistente y confirmaciones parciales
Reportes	Mismos filtros, cifras conciliadas, apertura real de XLS/PDF y estado HTTP con permiso insuficiente
Infraestructura/configuración	Salud externa, TLS, CORS, login y operación representativa tras reinicio/rollback

La secuencia de liberación recomendable es preparar datos de prueba y compatibilidad, verificar ambos consumidores, publicar imágenes inmutables, aplicar el despliegue controlado y registrar la comprobación funcional. Las mejoras de capacidad deben basarse en tiempos, memoria, volumen y consultas medidos. La arquitectura objetivo y sus migraciones propuestas se desarrollan en el plan de escalabilidad.

11. Capacidades verificables y referencias de soporte

11.1 Alcance implementado y validación por ambiente

La siguiente tabla orienta la aceptación y el mantenimiento. “Implementado” significa presente en el repositorio revisado; la versión activa de cada EC2 y la ejecución de integraciones se confirman en el ambiente antes de atribuirles ese estado.

Aspecto	Estado documentado al corte
Frontend y API HTTPS	Publicados y comprobados por sus URLs
CORS y comunicación entre componentes	Origen explícito aplicado; preflight verificado
Control de acceso	JWT, estado de usuario y permisos dinámicos implementados
Esquema versionado	Migraciones V1–V21; confirmar versión aplicada por ambiente
Reportes PDF	Exportador y endpoints implementados; requiere evidencia de conciliación con datos operativos
Exportación Excel	XLS binario implementado; `.xlsx` y CSV de reportes no admitidos; alinear selección frontend
2FA	Código por correo implementado en backend y flujo frontend; confirmar Gmail y V21 en el entorno
Correo y almacenamiento externo	Integraciones disponibles; entrega/carga real debe acreditarse por separado
Respaldo externo, incremental y PITR	No automatizados por la infraestructura actual
Alta disponibilidad	No implementada; evolución definida como trabajo futuro